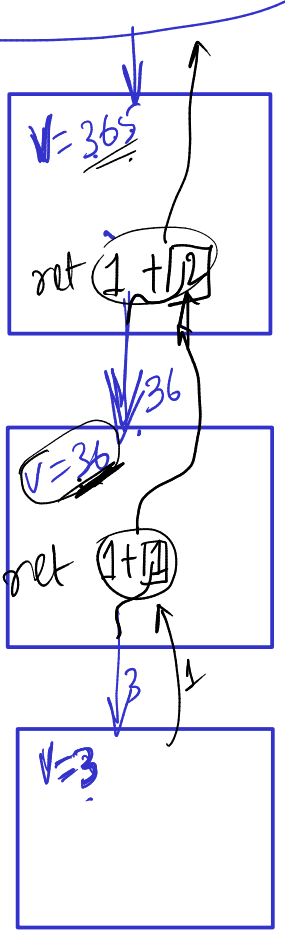
```
int cntDigits(int data)
{   int cnt=1;
    while(data/10)
    {  cnt++;
       data/=10;
    }
  return cnt;
}
```

data = ~~365~~ 36 3 cnt = 1 2 3

data = 8 cnt = 1



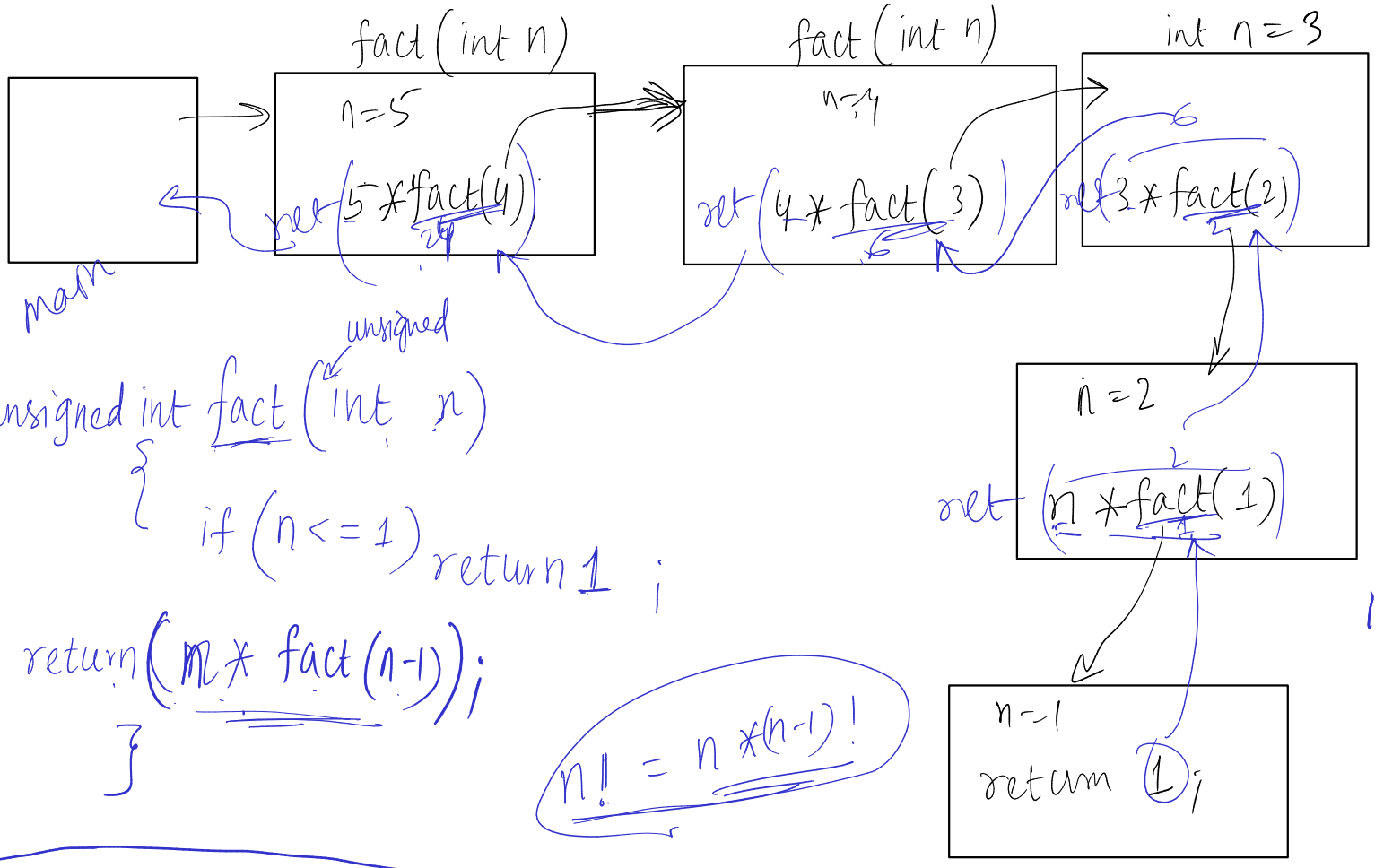

```
int cntDigits(int data){
    if (data<10) return 1;
    return (1+cntDigits(data/10));
}
```

```
unsigned int fact(int n)
{
  unsigned int r=1;
  while(n>=1){
     r=r*n;
     n--;
  }
  return r;
}
```

n! = n*(n-1)!

5! = 5x4x3x2x1

r=1
r=r*5;
r=r*4;
r=r*3;



create a function named fib

```
unsigned int fib(int);
```

which returns the fibonacci series Number, associated with a seq. no.

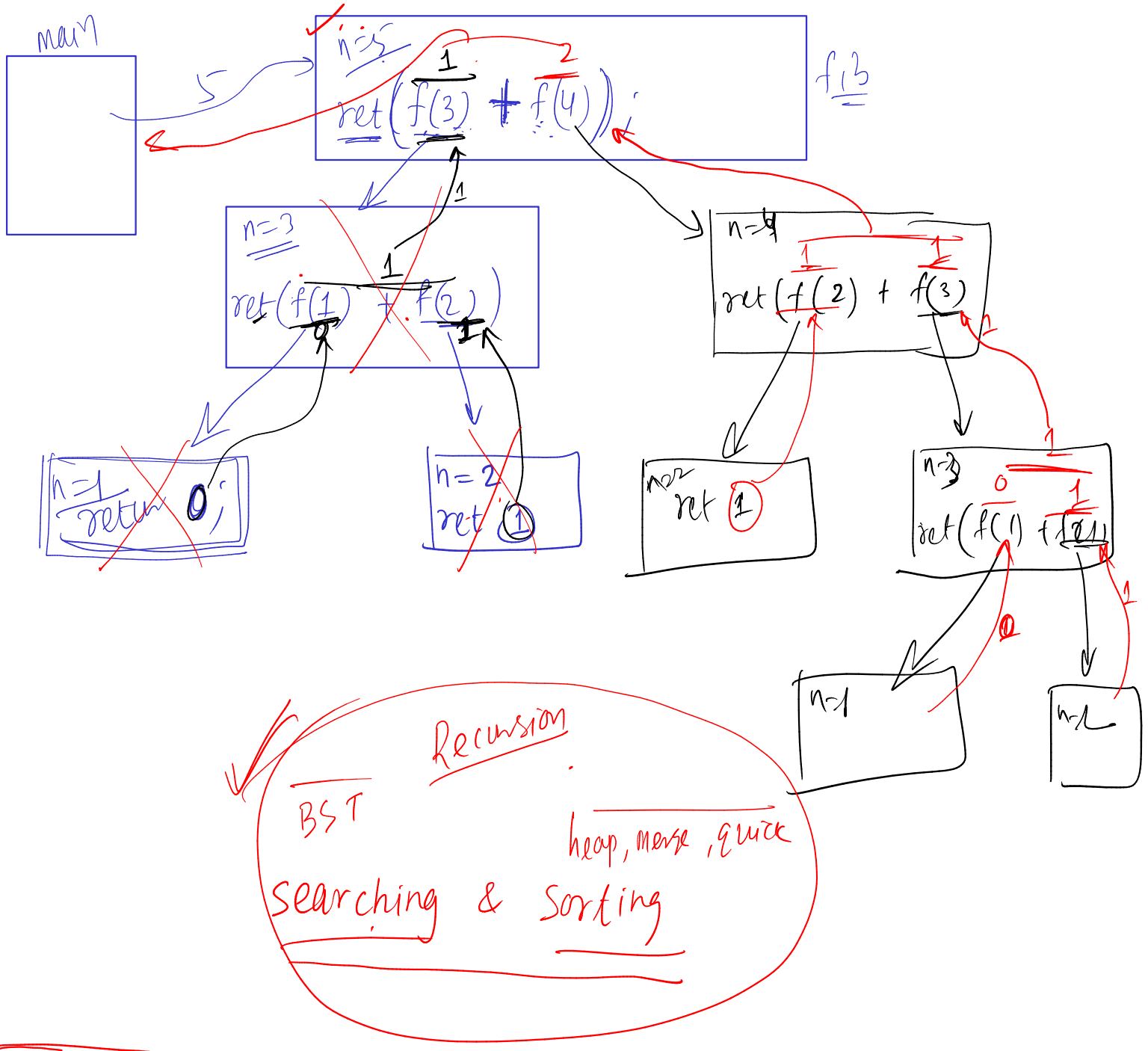
seq no	1	2	3	4	5	6	7	8	9	10	...
	0	1	1	2	3	5	8	13	21	34	...

n1=0 n2=1 n3=n1+n2

n1 n2 n3 = n1+n2

n1 n2 n3 = n1+n2

fib(n) = fib(n-1) + fib(n-2);

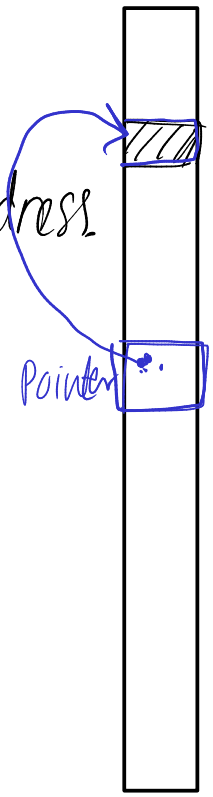


Rec, St. class, Pointer

Pointer / Indirection

What is Pointer?

→ A variable, capable to store Address



Why Pointer ?

→ TO Access the data Indirectly

Why to Access Data Indirectly?

⇒ Call by reference

⇒ Return multiple Data from function

⇒ Array & String

⇒ Dyn. mem. Allocation

⇒ Call back function (using function Pointer)

⇒ Data structure

⇒ Dyn (Runtime) linkage

etc...

⇒ Read the topic at Page no 75.

" " Chapt. Storage class

St_class - 75

Read

Pointer Basics

Pointer Declaration: Data-type *ptr_name; (* : Declarator)

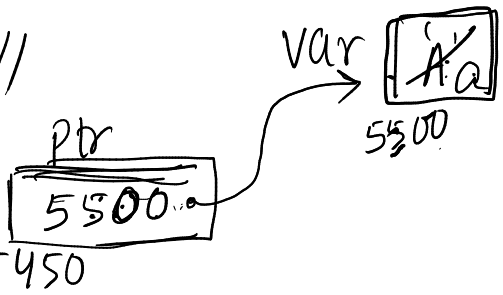
Assignment : ptr_name = ADDR;

Indirection: (*ptr_name) = Data; (* : operator)

var2 = (*ptr_name);

```
main() {  char var = 'A'; //
          char *ptr;
          ptr = &var;
          *ptr = var + 32; // *ptr = *ptr + 32;
          printf("var = %c  %c", var, *ptr);
}
```

reference operator



output:
a a